

Ciência dos Dados I: Análise Exploratória

Aula 03: Paradigmas de Programação e Sistemas de Tipagem

Prof. Dr. Ney Lemke

UNESP – Instituto de Biociências de Botucatu

Versão 2026

Sumário da Aula

O que é um Paradigma de Programação?

Sistema de Tipagem em Python

Exemplo Comparativo: Imperativo vs. Funcional

Classificação dos Paradigmas

Um paradigma representa uma forma fundamental de estruturar o raciocínio computacional:

- **Imperativo / Procedural:** Foco no *como fazer* (passo a passo de instruções e alteração de estado na memória). Ex: C, Fortran, Pascal.
- **Funcional:** Foco no *o que calcular* (composição de funções puras, ausência de efeitos colaterais e dados imutáveis). Ex: Haskell, Lisp, Elixir.
- **Orientado a Objetos (OOP):** Agrupamento de dados (atributos) e comportamentos (métodos) em entidades coesas (classes e objetos). Ex: Java, C++, Smalltalk.

Python é Multiparadigma

Python adota pragmatismo: permite combinar o melhor de cada paradigma conforme a necessidade da análise de dados!

Dinâmica, Forte e Type Hints

Tipagem Dinâmica

O tipo da variável é inferido em tempo de execução. Uma mesma variável pode referenciar diferentes tipos.

Tipagem Forte

Python não realiza coerção implícita de tipos incompatíveis (ex: `'10' + 5` resulta em `TypeError`).

Type Hints (PEP 484 até 2026)

Anotações explícitas de tipo aumentam a legibilidade e evitam bugs em pipelines complexos de dados.

Soma dos Quadrados dos Pares

Estilo Imperativo (Loops e Mutação)

```
soma = 0
for x in range(1, 11):
    if x % 2 == 0:
        soma += x ** 2
print(soma)
```

Estilo Funcional (Composição)

```
pares = filter(lambda x: x % 2 == 0, range(1, 11))
quadrados = map(lambda x: x ** 2, pares)
soma = sum(quadrados)
print(soma)
```

Na Prática de Dados

Pandas e PySpark utilizam princípios funcionais intensamente (transformações encadeadas sem alteração dos dados originais).

- Paradigmas moldam como expressamos transformações de dados.
- Tipagem forte garante integridade e type hints documentam expectativas.
- **Prática:** Expressões lógicas, casting explícito e funções lambda.